

Module 5.2: Cassandra

Contents

Cassandra	1
Cassandra Data Model	2
Cassandra Clients.....	3
HBase vs Cassandra	3

Cassandra

Apache Cassandra is an open-source, distributed NoSQL database designed to handle large amounts of data across multiple servers without a single point of failure. It provides high availability, fault tolerance, and horizontal scalability, making it suitable for big data and real-time applications.

- Developed originally by Facebook.
- Distributed and decentralized database.
- Supports large-scale data storage.
- Provides high availability and fault tolerance.
- Suitable for real-time applications.

Features of Cassandra

- **Distributed Architecture** – Data is distributed across multiple nodes.
- **High Availability** – No single point of failure in the system.
- **Fault Tolerance** – Data remains available even during node failures.
- **Linear Scalability** – Capacity increases by adding more nodes.
- **Peer-to-Peer Design** – All nodes have equal responsibilities.
- **Automatic Replication** – Data is replicated across multiple nodes.
- **Fast Write Performance** – Optimized for high-speed data insertion.
- **Flexible Schema** – Supports dynamic and evolving data structures.
- **Tunable Consistency** – Consistency level can be adjusted as needed.
- **Multi-Datacenter Support** – Supports deployment across multiple locations.

Cassandra Architecture

Cassandra follows a distributed peer-to-peer architecture where all nodes are equal and communicate directly with each other. There is no master server.

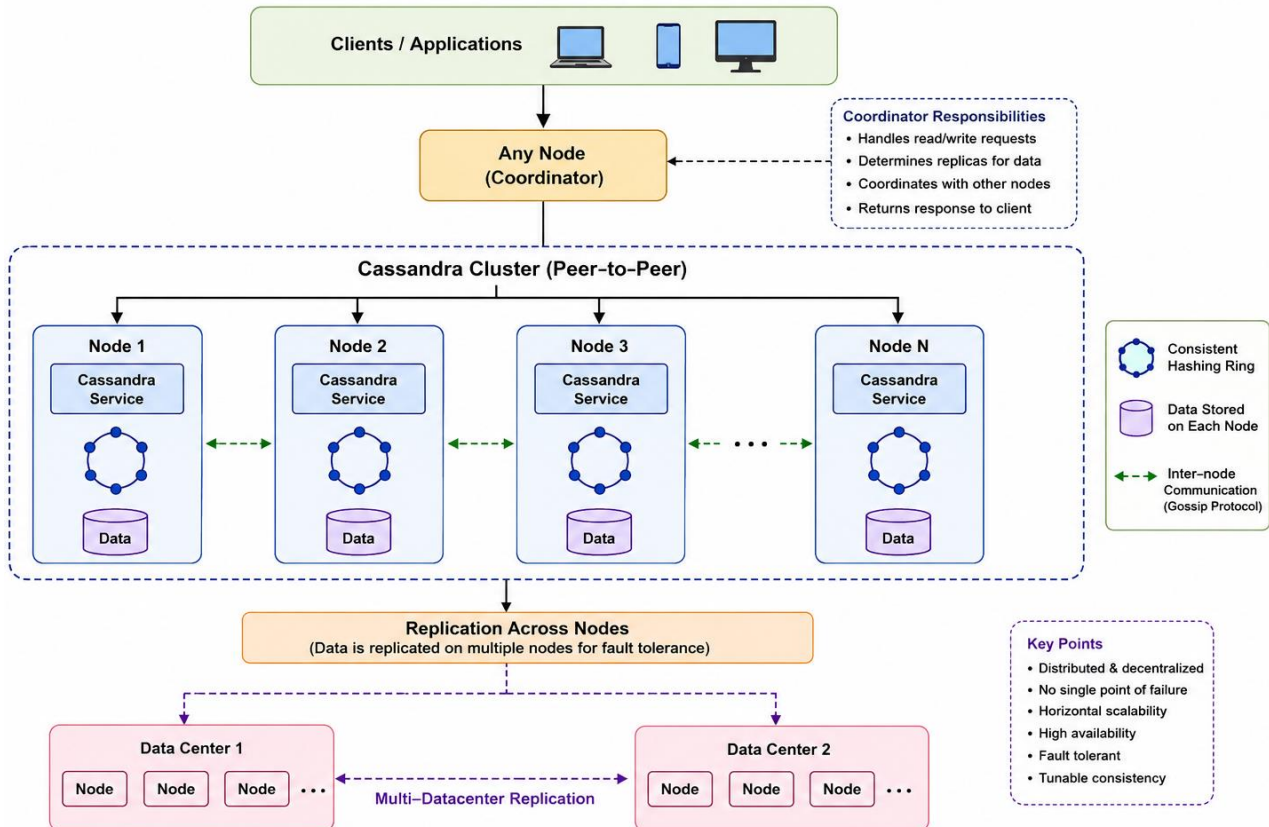
Working

1. Client sends a request to any node.
2. The receiving node acts as the coordinator.
3. Data is distributed across nodes using partitioning.
4. Replicas store copies of the data.
5. Nodes communicate to maintain consistency.
6. Requested data is returned to the client.

Key Elements of Cassandra Architecture

- **Client/Application** – Sends read and write requests to the Cassandra cluster.
- **Coordinator Node** – Receives client requests and coordinates communication with other nodes.
- **Cluster** – A collection of interconnected Cassandra nodes.
- **Nodes** – Individual servers that store and manage data.
- **Partitioning** – Distributes data across nodes using a partition key.
- **Replication** – Stores copies of data on multiple nodes for reliability.
- **Gossip Protocol** – Enables nodes to exchange status and cluster information.
- **Data Centers** – Groups of nodes that can be distributed across different geographic locations.

CASSANDRA ARCHITECTURE

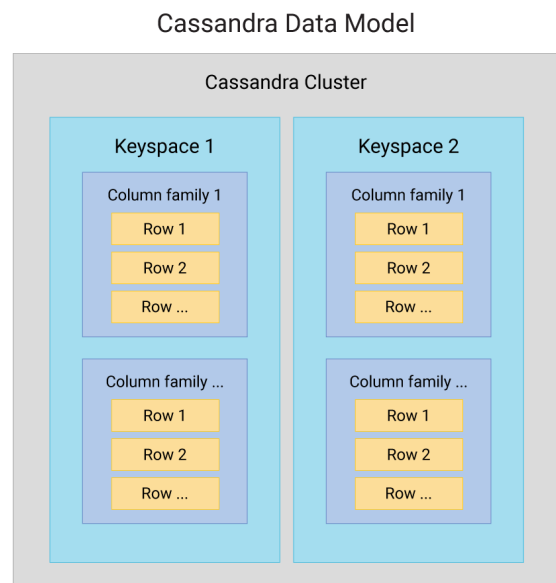


Advantages of Cassandra Architecture

- No single point of failure.
- High availability and fault tolerance.
- Horizontal scalability by adding more nodes.
- Fast read and write performance.
- Supports multi-data-center deployments.

Cassandra Data Model

- **Keyspace** – The top-level container in Cassandra, similar to a database in RDBMS.
- **Table (Column Family)** – Stores related data in rows and columns.
- **Row** – A collection of columns identified by a unique primary key.
- **Column** – Basic unit of data containing a name, value, and timestamp.
- **Primary Key** – Uniquely identifies rows and determines data distribution.
- **Partition Key** – Decides the node where data is stored.
- **Clustering Column** – Defines the sorting order of data within a partition.
- **Wide Rows** – Rows that can contain a large number of columns.
- **Schema-Free Design** – Allows flexible handling of large and evolving datasets.



Cassandra Clients

1) CQL Shell (cqlsh)

- Command-line interface used to interact with Cassandra.
- Supports creating keyspaces, tables, and executing queries.
- Similar to SQL shells in relational databases.
- Useful for database administration and testing.

2) Java Driver

- Official driver for connecting Java applications to Cassandra.
- Provides high-performance query execution.
- Supports asynchronous operations and connection pooling.
- Widely used in enterprise applications.

3) Python Driver

- Allows Python applications to communicate with Cassandra.
- Supports CQL query execution and data retrieval.
- Easy integration with data analytics and machine learning projects.
- Commonly used in scripting and automation tasks.

4) Other Language Drivers

- Node.js Driver for JavaScript applications.
- C# Driver for .NET applications.
- Go Driver for Go-based systems.
- PHP and Ruby Drivers for web application development.

HBase vs Cassandra

Feature	HBase	Cassandra
Developer	Apache Software Foundation	Apache Software Foundation (originally Facebook)
Database Type	Column-oriented NoSQL database	Column-family NoSQL database
Architecture	Master-Slave Architecture	Peer-to-Peer Architecture
Master Node	Uses HMaster	No master node
Storage	Runs on HDFS	Uses its own distributed storage
Scalability	Horizontal scalability	Horizontal scalability
Fault Tolerance	Achieved through HDFS replication	Achieved through data replication across nodes
Read Performance	Good	Very high
Write Performance	High	Very high
Consistency	Strong consistency	Tunable consistency
Availability	High	Very high
Data Distribution	Regions managed by Region Servers	Partitioning using consistent hashing
Query Language	HBase Shell, Java API	CQL (Cassandra Query Language)
Hadoop Integration	Native integration with Hadoop	Integration through connectors
Best Use Cases	Batch processing, analytics, Hadoop ecosystems	Real-time applications, high-availability systems